EC-110 Computing Fundamentals Experiment # 8

Arithmetic Operations.

1. Objective/s:

- Order of Precedence
- Expressions
- Type Conversion
- Lab Task

2. Order of Precedence

Order of Precedence is the principal by which order of execution of an arithmetic expression with different operators is decided. C++ has multiple arithmetic operators which include

- + addition
- - subtraction
- * multiplication
- / division
- % modulus operator

All the above mentioned operators except for % can be used with integral and floating-point data types

Any expression (explained later in manual) having multiple operators e.g. 3 * 7 - 6 + 2 * 5 / 4 + 6 will be executed by following these points

- All operations inside of () are evaluated first
- *, /, and % are at the same level of precedence and are evaluated next •

+ and – have the same level of precedence and are evaluated last

Hence the example expression 3 * 7 - 6 + 2 * 5 / 4 + 6 becomes (((3 * 7) – 6) + ((2 * 5) / 4)) + 6

3. Expressions

An expression is a sequence of operators and operands that specifies a computation.

We will learn about Intergral/floating or mixed expression

3.1.Integral/Floating Expression

- If all operands are integers
  - Expression is called an integral expression
    - Yields an integral result
    - Example: 2 + 3 * 5
- If all operands are floating-point
  - Expression is called a floating-point expression
    - Yields a floating-point result
    - Example: 12.8 * 17.5 - 34.50

3.2. Mixed Expression

- Mixed expression:
  - Has operands of different data types
  - Contains integers and floating-point
- Examples of mixed expressions:

    2 + 3.5

    6 / 4 + 3.9

    5.4 * 2 – 13.6 + 18 / 2

3.3.Evaluation Rule

- If operator has same types of operands
  - Evaluated according to the type of the operands
- If operator has both types of operands
  - Integer is changed to floating-point
  - Operator is evaluated
  - Result is floating-point
- Entire expression is evaluated according to precedence rules

4. Type Conversion (Casting)

4.1.Implicit Type Conversion

• when value of one type is automatically changed to another type

4.2.Explicit Type Conversion

• provides explicit type conversion

static_cast<desired dataTypeName>(expression)

EXAMPLE 2-9

Expression	Evaluates to
`static_cast<int>(7.9)`	7
`static_cast<int>(3.3)`	3
`static_cast<double>(25)`	25.0
`static_cast<double>(5+3)`	= `static_cast<double>(8)` = 8.0
`static_cast<double>(15) / 2`	= 15.0 / 2 (because `static_cast<double>(15)` = 15.0) = 15.0 / 2.0 = 7.5
`static_cast<double>(15 / 2)`	= `static_cast<double>(7)` (because 15 / 2 = 7) = 7.0
`static_cast<int>(7.8 + static_cast<double>(15) / 2)`	= `static_cast<int>(7.8+7.5)` = `static_cast<int>(15.3)` = 15
`static_cast<int>(7.8 + static_cast<double>(15 / 2))`	= `static_cast<int>(7.8 + 7.0)` = `static_cast<int>(14.8)` = 14

Department of CE HITEC University, Taxila

BSCE – 1st Semester Lab – 08 Fall 2021

5. Tasks

1. Write and run a program that performs the following steps:
   - Assigning value to a Fahrenheit temperature f.
   - Calculating the equivalent Celsius temperature C using the formula:
     C = (����) x (f –32).
   - Displaying the Celsius temperature C.

2. Write and run a program that takes radius of a circle from user and displays the area and circumference of the circle on screen.

3. Write and run a program that gets two numbers from user and displays their

- Addition
- Subtraction
- Multiplication
- Division
- Remainder